



UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação

Programa de Pós-Graduação em Engenharia da Computação

Disciplina: Redes Sem Fio

**MEDIÇÃO E ANÁLISE DE RSSI EM REDES WI-FI:
COMPARAÇÃO ENTRE MEDIDAS REAIS (ESP32) E O MODELO DE
PROPAGAÇÃO LOG-DISTANCE**

Álison de Oliveira Venâncio

Ayrton Finicelli Lemes

Matheus Serrão Uchôa

Victor Ronald

Manaus – AM

2026

Álison de Oliveira Venâncio

Ayrton Finicelli Lemes

Matheus Serrão Uchôa

Victor Ronald

**MEDIÇÃO E ANÁLISE DE RSSI EM REDES WI-FI:
COMPARAÇÃO ENTRE MEDIDAS REAIS (ESP32) E O MODELO DE
PROPAGAÇÃO LOG-DISTANCE**

Relatório técnico apresentado à disciplina de Redes Sem Fio do Programa de Pós-Graduação em Engenharia da Computação da Universidade Federal do Amazonas (UFAM), como requisito parcial de avaliação.

Professor: Prof. Celso Barbosa Carvalho

RESUMO

Este relatório descreve uma atividade prática de medição e análise do RSSI (*Received Signal Strength Indicator*) de uma rede Wi-Fi em ambiente residencial real, comparando os dados coletados com o modelo teórico de propagação *log-distance*. Utilizou-se a plataforma ESP32 para coletar 30 amostras de RSSI em cada uma de nove distâncias do ponto de acesso (de 1.0 m a 5.0 m, em passos de 0.5 m), totalizando 270 medições. Os dados foram organizados em arquivo CSV por um *script* Python e processados para o cálculo das médias por distância. Em paralelo, implementou-se o modelo *log-distance* parametrizado com potência de transmissão do AP $P_t = 20$ dBm, expoente de perda $n = 3,0$ e desvio-padrão de *shadowing* $\sigma = 4$ dB, gerando 30 amostras simuladas por distância. A comparação mostrou boa aderência entre teoria e prática (RMSE de 1.89 dB entre as médias), com expoente de perda efetivo de $n_{ef} = 3,23$ estimado por regressão sobre os dados reais. As discrepâncias observadas são explicadas por multipercurso, obstruções e ruído, reforçando a importância do termo estocástico de *shadowing* para representar cenários reais.

Palavras-chave: RSSI. Wi-Fi. ESP32. Modelo Log-Distance. Propagação. *Shadowing*. IEEE 802.11.

SUMÁRIO

1	INTRODUÇÃO	4
2	OBJETIVOS	5
2.1	Objetivo Geral	5
2.2	Objetivos Específicos	5
3	FUNDAMENTAÇÃO TEÓRICA	6
3.1	RSSI e Qualidade do Sinal Wi-Fi	6
3.2	Modelo de Propagação Log-Distance	6
3.3	Multipercurso e Sombreamento	7
3.4	Plataforma ESP32	7
4	MATERIAIS E METODOLOGIA	8
4.1	Materiais	8
4.2	Local da Coleta	8
4.3	Procedimento	8
5	IMPLEMENTAÇÃO	9
5.1	Firmware ESP32 (envia_rssi.ino)	9
5.2	Coleta Serial (coleta_dados.py)	10
5.3	Modelo Log-Distance (log_distancia.py)	12
5.4	Gráfico Comparativo (grafico_comparativo.py)	14
6	RESULTADOS	17
6.1	Parte 1 — Dados Coletados	17
6.2	Parte 2 — Gráfico dos Dados Reais	17
6.3	Parte 3 — Simulação Teórica	18
6.4	Parte 4 — Gráfico Comparativo	19
7	ANÁLISE E DISCUSSÃO	20
7.1	Diferenças entre Teoria e Prática	20
7.2	Fatores que Explicam as Discrepâncias	20
7.3	Importância da Modelagem Estatística	20
8	CONCLUSÃO	22
	REFERÊNCIAS	23

1 INTRODUÇÃO

O monitoramento da intensidade do sinal recebido — expressa pelo RSSI (*Received Signal Strength Indicator*) — é um dos parâmetros mais relevantes em redes sem fio IEEE 802.11 (Wi-Fi). Além de indicar a qualidade do enlace, o RSSI fundamenta aplicações de localização *indoor*, planejamento de cobertura, *handover* e detecção de interferências (1).

A relação entre a potência recebida e a distância ao transmissor é descrita por modelos de propagação. O mais difundido em ambientes internos é o modelo *log-distance*, que combina um decaimento determinístico proporcional ao logaritmo da distância com um termo estocástico de *shadowing* (sombreamento) (2, 3). Entretanto, ambientes reais introduzem obstáculos, reflexões e múltiplos caminhos que afastam as medidas das previsões teóricas.

Esta atividade tem por objetivo coletar amostras reais de RSSI com a plataforma ESP32 em função da distância a um ponto de acesso (AP) e compará-las com o modelo *log-distance*, discutindo as diferenças entre teoria e prática e a importância da modelagem estatística.

2 OBJETIVOS

2.1 OBJETIVO GERAL

Medir o RSSI de uma rede Wi-Fi em função da distância ao ponto de acesso, em ambiente residencial real, e comparar os resultados com o modelo teórico de propagação *log-distance*.

2.2 OBJETIVOS ESPECÍFICOS

- a) Implementar *firmware* ESP32 para enviar leituras de RSSI do AP conectado via porta serial;
- b) Coletar 30 amostras de RSSI por distância, iniciando a ≈ 1 m e afastando-se em passos de 0.5 m até, no mínimo, 3 m;
- c) Organizar as leituras em arquivo CSV e calcular a média por distância;
- d) Gerar gráfico dos dados reais (distância $\times$ RSSI médio);
- e) Implementar o modelo log-distance com parâmetros realistas e gerar 30 amostras simuladas por distância;
- f) Comparar, em um único gráfico, os dados reais com a curva teórica e discutir as discrepâncias.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 RSSI E QUALIDADE DO SINAL WI-FI

O RSSI mede a potência do sinal recebido, expressa em dBm (decibéis referidos a 1 mW). Valores típicos e a respectiva qualidade percebida são apresentados na Tabela 1.

Tabela 1 – Faixas de RSSI e qualidade do sinal Wi-Fi.

RSSI (dBm)	Qualidade	Descrição
> -50	Excelente	Sinal muito forte
-50 a -60	Muito bom	Adequado para <i>streaming</i> HD
-60 a -70	Bom	Navegação satisfatória
-70 a -80	Razoável	Desempenho reduzido
-80 a -90	Fraco	Instabilidade frequente
< -90	Inutilizável	Conexão não confiável

Fonte: Adaptado de 2.

3.2 MODELO DE PROPAGAÇÃO LOG-DISTANCE

A perda de percurso (*path loss*) média cresce logaritmicamente com a distância. O modelo *log-distance* é dado pela Equação 3.1 (2, 3):

$$PL(d) = PL(d_0) + 10 n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (3.1)$$

onde $PL(d)$ é a perda à distância d ; $PL(d_0)$ é a perda na distância de referência d_0 (tipicamente 1 m); n é o expoente de perda de percurso; e $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ é o termo gaussiano de *shadowing*, em dB.

A potência recebida (RSSI) relaciona-se com a potência transmitida P_t pela Equação 3.2:

$$RSSI(d) = P_t - PL(d) = \underbrace{[P_t - PL(d_0)]}_{RSSI(d_0)} - 10 n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (3.2)$$

O expoente n depende do ambiente: $n = 2$ no espaço livre e tipicamente $n = 2,7$ a $4,3$ em ambientes internos com paredes e mobiliário (2). O desvio-padrão do *shadowing* situa-se, em geral, entre 3 dB e 12 dB, conforme o grau de obstrução (3, 4).

3.3 MULTIPERCURSO E SOMBREAMENTO

Em ambientes fechados, o sinal alcança o receptor por múltiplos caminhos (reflexões em paredes, piso, teto e mobiliário). A soma vetorial dessas componentes pode ser construtiva ou destrutiva, gerando flutuações de pequena escala (*fading*) sobrepostas à tendência de grande escala descrita pelo termo X_σ (4). Por isso, medidas reais raramente decaem de forma estritamente monótona com a distância.

3.4 PLATAFORMA ESP32

O ESP32-WROOM-32, da Espressif Systems, integra em um *System-on-Chip* um processador *dual-core* Xtensa LX6 a 240 MHz com Wi-Fi 802.11 b/g/n (2.4 GHz) (5). A API `WiFi.RSSI()` retorna o RSSI, em dBm, do AP atualmente conectado (6), o que o torna adequado para campanhas de medição de baixo custo.

4 MATERIAIS E METODOLOGIA

4.1 MATERIAIS

- **Hardware:** placa ESP32-WROOM-32 *Dev Module*;
- **Ponto de acesso:** roteador residencial 2.4 GHz, potência de transmissão $P_t \approx 20$ dBm (100 mW EIRP), dentro do limite de radiação restrita da ANATEL (7);
- **IDE:** Arduino IDE 2.x com suporte ESP32 (Espressif 3.x);
- **Linguagens:** C++ (Arduino) e Python 3.11;
- **Bibliotecas Python:** `pyserial`, `numpy`, `matplotlib`.

4.2 LOCAL DA COLETA

A campanha foi realizada em uma residência localizada na Travessa Castelo Branco, bairro Compensa, Manaus/AM. As medições ocorreram ao longo de um corredor interno conectado à sala, com paredes de alvenaria e mobiliário típico, condição representativa de um ambiente residencial urbano amazônico densamente ocupado por redes Wi-Fi vizinhas na faixa de 2.4 GHz.

4.3 PROCEDIMENTO

1. Carga do *firmware* `envia_rssi.ino` no ESP32, conectando ao AP sob medição;
2. Posicionamento do ESP32 a 1.0 m do AP e coleta de 30 amostras de RSSI com o *script* `coleta_dados.py`;
3. Afastamento em passos de 0.5 m (1.0, 1.5, ..., 5.0 m), repetindo a coleta de 30 amostras em cada ponto — totalizando 9 distâncias e 270 medições;
4. Cálculo da média e do desvio-padrão por distância (`grafico_valor_medio.py`);
5. Simulação do modelo log-distance (`log_distancia.py`) com $P_t = 20$ dBm, $n = 3,0$ e $\sigma = 4$ dB;
6. Comparação entre dados reais e curva teórica (`grafico_comparativo.py`).

5 IMPLEMENTAÇÃO

5.1 FIRMWARE ESP32 (ENVIA_RSSI.INO)

O *firmware* conecta o ESP32 ao AP e envia o RSSI do enlace pela serial a cada 200 ms, uma leitura por linha.

Listing 5.1 – Firmware ESP32 — envio periódico do RSSI do AP conectado.

```

1  /*
2  * envia_rssi.ino
3  * -----
4  * Conecta o ESP32 a um ponto de acesso WiFi e envia, pela porta serial,
5  * a leitura de RSSI (Received Signal Strength Indicator) do AP conectado
6  * a cada 200 ms. Cada linha contem apenas o valor inteiro de RSSI em dBm,
7  * facilitando a captura pelo script Python "coleta_dados.py".
8  *
9  * Disciplina: Redes Sem Fio - PPGEEC/UFAM
10 * Plataforma: ESP32-WROOM-32 (Espressif), Arduino IDE 2.x
11 * -----
12 */
13
14 #include "WiFi.h"
15
16 // -- Credenciais do ponto de acesso sob medicaao -----
17 const char* SSID_AP = "RSSI_LAB_2G"; // ajuste para o SSID do seu AP
18 const char* SENHA_AP = "senha_do_ap"; // ajuste para a senha do seu AP
19
20 // Intervalo entre leituras (ms)
21 const unsigned long INTERVALO_MS = 200;
22
23 void conectarWiFi() {
24     Serial.printf("Conectando_a_%s...\n", SSID_AP);
25     WiFi.mode(WIFI_STA);
26     WiFi.begin(SSID_AP, SENHA_AP);
27
28     unsigned long inicio = millis();
29     while (WiFi.status() != WL_CONNECTED) {
30         delay(300);
31         Serial.print(".");
32         // tenta reconectar caso demore demais
33         if (millis() - inicio > 15000) {
34             Serial.println("\nFalha_na_conexao._Tentando_novamente...");
35             WiFi.disconnect();
36             delay(500);
37             WiFi.begin(SSID_AP, SENHA_AP);
38             inicio = millis();
39         }
40     }
41     Serial.println();
42     Serial.printf("Conectado!_IP:_%s_|_BSSID:_%s\n",
43         WiFi.localIP().toString().c_str(),

```

```

44         WiFi.BSSIDstr().c_str());
45     }
46
47     void setup() {
48         Serial.begin(115200);
49         delay(200);
50         conectarWiFi();
51         Serial.println("#_Iniciando_envio_de_RSSI_(dBm) ...");
52     }
53
54     void loop() {
55         if (WiFi.status() != WL_CONNECTED) {
56             conectarWiFi();
57         }
58
59         long rssi = WiFi.RSSI();    // RSSI do AP atualmente conectado (dBm)
60
61         // Envia apenas o numero inteiro, uma leitura por linha.
62         Serial.println(rssi);
63
64         delay(INTERVALO_MS);
65     }

```

5.2 COLETA SERIAL (COLETA_DADOS.PY)

O *script* lê a serial, agrupa 30 amostras por distância informada pelo operador e grava em `medicoes_rssi.csv`.

Listing 5.2 – Coleta e organização das amostras de RSSI por distância.

```

1  """
2  coleta_dados.py
3  -----
4  Captura as leituras de RSSI enviadas pelo ESP32 (firmware envia_rssi.ino) pela
5  porta serial e organiza as amostras por distancia do ponto de acesso.
6
7  Procedimento de coleta:
8      1. Posicione o ESP32 a ~1,0 m do AP e rode o script informando a distancia.
9      2. O script coleta N (=30) amostras de RSSI e grava em medicoes_rssi.csv.
10     3. Afaste-se ~0,5 m e repita, ate >= 3,0 m (ou o limite do ambiente).
11
12 Saída: medicoes_rssi.csv com colunas [distancia_m, amostra, rssi_dbm].
13
14 Disciplina: Redes Sem Fio - PPGEEC/UFAM
15 -----
16 """
17
18 import csv
19 import os
20 import sys
21 import time
22
23 import serial    # pip install pyserial
24

```

```

25 PORTA_SERIAL = "/dev/ttyUSB0" # Windows: "COM6"; Linux: "/dev/ttyUSB0"
26 BAUDRATE = 115200
27 N_AMOSTRAS = 30 # amostras por distancia
28 ARQUIVO_CSV = "medicoes_rssi.csv"
29
30
31 def ler_rssi(ser):
32     """Le uma linha da serial e retorna o RSSI (int) ou None se invalida."""
33     linha = ser.readline().decode("utf-8", errors="ignore").strip()
34     if not linha or linha.startswith("#"):
35         return None
36     try:
37         valor = int(linha)
38     except ValueError:
39         return None
40     # filtra leituras fora da faixa fisica plausivel
41     if -100 <= valor <= -10:
42         return valor
43     return None
44
45
46 def coletar(distancia, ser):
47     """Coleta N_AMOSTRAS leituras validas de RSSI para a distancia dada."""
48     print(f"\n>> Coletando {N_AMOSTRAS} amostras a {distancia:.1f}m do AP...")
49     amostras = []
50     while len(amostras) < N_AMOSTRAS:
51         rssi = ler_rssi(ser)
52         if rssi is not None:
53             amostras.append(rssi)
54             print(f"_{len(amostras):02d}/{N_AMOSTRAS}_RSSI={rssi}_dBm")
55     media = sum(amostras) / len(amostras)
56     print(f"_{media:.1f}_m:_{media:.2f}_dBm")
57     return amostras
58
59
60 def salvar(distancia, amostras):
61     """Acrescenta as amostras ao CSV (cria cabecalho se necessario)."""
62     novo = not os.path.exists(ARQUIVO_CSV)
63     with open(ARQUIVO_CSV, "a", newline="", encoding="utf-8") as f:
64         w = csv.writer(f)
65         if novo:
66             w.writerow(["distancia_m", "amostra", "rssi_dbm"])
67         for k, r in enumerate(amostras, start=1):
68             w.writerow([f"{distancia:.1f}", k, r])
69     print(f"_{ARQUIVO_CSV}.")
70
71
72 def main():
73     try:
74         ser = serial.Serial(PORTA_SERIAL, BAUDRATE, timeout=2)
75     except serial.SerialException as e:
76         print(f"Erro ao abrir {PORTA_SERIAL}: {e}")
77         sys.exit(1)
78
79     time.sleep(2) # aguarda o ESP32 reiniciar/estabilizar
80     print(f"Conectado a {PORTA_SERIAL}. Pressione Ctrl+C para encerrar.")
81

```

```

82     try:
83         while True:
84             entrada = input("\nDistancia_do_AP_em_metros_(ENTER_vazio_p/_sair):_")
85             if entrada.strip() == "":
86                 break
87             distancia = float(entrada.replace(", ", "."))
88             amostras = coletar(distancia, ser)
89             salvar(distancia, amostras)
90     except KeyboardInterrupt:
91         print("\nColeta_encerrada_pelo_usuario.")
92     finally:
93         ser.close()
94         print("Porta_serial_fechada.")
95
96
97 if __name__ == "__main__":
98     main()

```

5.3 MODELO LOG-DISTANCE (LOG_DISTANCIA.PY)

Implementa a Equação 3.2, gerando 30 amostras simuladas de RSSI por distância e a respectiva média.

Listing 5.3 – Simulação do modelo log-distance (30 amostras por distância).

```

1  """
2  log_distancia.py
3  -----
4  Parte 3 - Simulacao teorica (Modelo Log-Distance de Potencia de Sinal).
5
6  Modelo:
7      PL(d)   = PL(d0) + 10 * n * log10(d / d0) + X_sigma,   X_sigma ~ N(0, sigma)
8      RSSI(d) = Pt - PL(d)
9
10 Para cada distancia sao geradas 30 amostras de RSSI simuladas; em seguida,
11 calcula-se a media por distancia e plota-se a curva teorica.
12
13 Parametros usados (proximos da realidade):
14     Pt      = 20 dBm   -> potencia de transmissao tipica de roteador/ONT
15                      residencial 2,4 GHz (100 mW EIRP).
16     d0      = 1 m      -> distancia de referencia.
17     PL(d0)  = 59 dB    -> perda de referencia a 1 m (perda de espaco livre a
18                      2,4 GHz, ~40 dB, somada a perdas de implementacao:
19                      eficiencia da antena do ESP32, casamento e obstaculos
20                      proximos). Assim Pt - PL(d0) = -39 dBm em d0.
21     n       = 3.0      -> expoente de perda (faixa 2,3 a 3,5 - ambiente interno).
22     sigma   = 4 dB     -> desvio-padrao do shadowing (faixa 3 a 5 dB).
23
24 Saidas:
25     images/rssi_medio_teorico.pdf/png
26     medicoes_simuladas.csv ([distancia_m, amostra, rssi_dbm])
27
28 Disciplina: Redes Sem Fio - PPGEEC/UFAM
29 -----

```

```

30 """
31
32 import csv
33
34 import matplotlib
35
36 matplotlib.use("Agg")
37 import matplotlib.pyplot as plt
38 import numpy as np
39
40 # -- Parametros do modelo -----
41 Pt = 20.0          # potencia de transmissao do AP (dBm)
42 d0 = 1.0           # distancia de referencia (m)
43 PL_d0 = 59.0       # perda de caminho na distancia de referencia (dB)
44 n = 3.0            # expoente de perda de caminho
45 sigma = 4.0        # desvio-padrao do shadowing (dB)
46 N_AMOSTRAS = 30    # amostras simuladas por distancia
47
48 # Mesmas distancias da campanha de medicao real
49 DISTANCIAS = np.array([1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0])
50
51 rng = np.random.default_rng(7) # reprodutibilidade da simulacao
52
53
54 def rssi_simulado(distancias, n_amostras):
55     """Gera n_amostras de RSSI por distancia segundo o modelo log-distance."""
56     amostras = {}
57     for d in distancias:
58         pl = PL_d0 + 10.0 * n * np.log10(d / d0)
59         x_sigma = rng.normal(0.0, sigma, n_amostras)
60         rssi = Pt - pl + x_sigma
61         amostras[d] = rssi
62     return amostras
63
64
65 def main():
66     amostras = rssi_simulado(DISTANCIAS, N_AMOSTRAS)
67     media = np.array([amostras[d].mean() for d in DISTANCIAS])
68
69     # curva teorica "limpa" (sem ruido) para referencia
70     pl_curva = PL_d0 + 10.0 * n * np.log10(DISTANCIAS / d0)
71     rssi_curva = Pt - pl_curva
72
73     # -- grava amostras simuladas -----
74     with open("medicoes_simuladas.csv", "w", newline="", encoding="utf-8") as f:
75         w = csv.writer(f)
76         w.writerow(["distancia_m", "amostra", "rssi_dbm"])
77         for d in DISTANCIAS:
78             for k, r in enumerate(amostras[d], start=1):
79                 w.writerow([f"{d:.1f}", k, f"{r:.1f}"])
80     print("medicoes_simuladas.csv_gravado.")
81
82     # -- grafico -----
83     fig, ax = plt.subplots(figsize=(8.5, 5))
84
85     # nuvem de amostras simuladas
86     for d in DISTANCIAS:

```

```

87     ax.scatter(np.full(N_AMOSTRAS, d), amostras[d], s=12, alpha=0.25,
88                color="#9C27B0", zorder=2)
89     ax.scatter([], [], s=14, color="#9C27B0", alpha=0.5,
90                label="Amostras_simuladas_(30/dist.)")
91
92     ax.plot(DISTANCIAS, media, "o-", color="#6A1B9A", linewidth=2,
93             markersize=7, zorder=4, label="Media_simulada_por_distancia")
94     ax.plot(DISTANCIAS, rssi_curva, "--", color="#37474F", linewidth=1.6,
95             zorder=3, label="Curva_teorica_(sem_ruido)")
96
97     ax.set_xlabel("Distancia_do_AP_(m)", fontsize=11)
98     ax.set_ylabel("RSSI_(dBm)", fontsize=11)
99     ax.set_title("Modelo_Log-Distance_-_RSSI_simulado_em_funcao_da_distancia\n"
100                 f"Pt={Pt:.0f}_dBm, n={n}, sigma={sigma:.0f}_dB",
101                 fontsize=11, fontweight="bold")
102     ax.grid(alpha=0.35)
103     ax.legend(fontsize=9)
104     fig.tight_layout()
105     fig.savefig("images/rssi_medio_teorico.pdf")
106     fig.savefig("images/rssi_medio_teorico.png", dpi=150)
107     print("images/rssi_medio_teorico.pdf|png_gravado.")
108
109
110 if __name__ == "__main__":
111     main()

```

5.4 GRÁFICO COMPARATIVO (GRAFICO_COMPARATIVO.PY)

Sobrepõe os dados reais à curva teórica, ajusta o expoente n efetivo dos dados reais e quantifica a aderência (RMSE e MAE).

Listing 5.4 – Comparação entre dados reais e modelo log-distance.

```

1  """
2  grafico_comparativo.py
3  -----
4  Parte 4 - Comparacao e discussao.
5  Sobrepe, em um unico grafico:
6  - os dados reais (media do RSSI por distancia, com barras de desvio-padrao);
7  - a curva teorica do modelo log-distance (Parte 3).
8
9  Adicionalmente:
10 - ajusta o expoente de perda 'n' efetivo aos dados reais (regressao linear
11   de RSSI vs. log10(d));
12 - quantifica a aderencia teoria x pratica (RMSE e MAE das medias).
13
14 Este e o grafico comparativo entregue na atividade.
15
16 Disciplina: Redes Sem Fio - PPGEEC/UFAM
17 -----
18 """
19
20 import csv
21 from collections import defaultdict

```

```

22
23 import matplotlib
24
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27 import numpy as np
28
29 ARQUIVO = "medicoes_rssi.csv"
30
31 # -- Parametros do modelo teorico (iguais aos de log_distancia.py) -----
32 Pt = 20.0          # potencia de transmissao do AP (dBm)
33 d0 = 1.0           # distancia de referencia (m)
34 PL_d0 = 59.0       # perda de caminho na distancia de referencia (dB)
35 n_teorico = 3.0     # expoente de perda assumido
36 sigma = 4.0        # desvio-padrao do shadowing (dB)
37
38
39 def carregar(arquivo):
40     dados = defaultdict(list)
41     with open(arquivo, newline="", encoding="utf-8") as f:
42         for linha in csv.DictReader(f):
43             dados[float(linha["distancia_m"])] .append(int(linha["rssi_dbm"]))
44     return dict(sorted(dados.items()))
45
46
47 def main():
48     dados = carregar(ARQUIVO)
49     dist = np.array(list(dados.keys()))
50     media = np.array([np.mean(v) for v in dados.values()])
51     desvio = np.array([np.std(v, ddof=1) for v in dados.values()])
52
53     # -- curva teorica -----
54     d_cont = np.linspace(dist.min(), dist.max(), 200)
55     rssi_teorico = Pt - (PL_d0 + 10.0 * n_teorico * np.log10(d_cont / d0))
56     rssi_teorico_pts = Pt - (PL_d0 + 10.0 * n_teorico * np.log10(dist / d0))
57
58     # -- ajuste do n efetivo aos dados reais -----
59     # RSSI = a + b*log10(d), com b = -10*n -> n_efetivo = -b/10
60     x = np.log10(dist / d0)
61     b, a = np.polyfit(x, media, 1)          # media ~ b*x + a
62     n_efetivo = -b / 10.0
63     rssi_ajustado = a + b * np.log10(d_cont / d0)
64
65     # -- metricas de aderencia (media real x teorica) -----
66     erro = media - rssi_teorico_pts
67     rmse = float(np.sqrt(np.mean(erro ** 2)))
68     mae = float(np.mean(np.abs(erro)))
69
70     print(f"n_teorico_assumido: {n_teorico:.2f}")
71     print(f"n_efetivo_ajuste: {n_efetivo:.2f}")
72     print(f"RSSI(1_m)_real: {media[0]:.1f}_dBm")
73     print(f"RMSE(real_x_teorica): {rmse:.2f}_dB")
74     print(f"MAE(real_x_teorica): {mae:.2f}_dB")
75
76     # -- grafico comparativo -----
77     fig, ax = plt.subplots(figsize=(9, 5.5))
78

```

```

79 ax.plot(d_cont, rssi_teorico, "--", color="#37474F", linewidth=2,
80         label=f"Curva_teorica_log-distance_(n={n_teorico:.1f},_
81             f"$\\sigma$={sigma:.0f}_dB) ")
82 # faixa de incerteza +/- sigma do modelo
83 ax.fill_between(d_cont, rssi_teorico - sigma, rssi_teorico + sigma,
84                color="#90A4AE", alpha=0.25,
85                label=f"Faixa_teorica_+/-_\\sigma$_({sigma:.0f}_dB) ")
86 ax.plot(d_cont, rssi_ajustado, ":", color="#E65100", linewidth=1.8,
87         label=f"Ajuste_aos_dados_reais_(n_efetivo={n_efetivo:.2f}) ")
88
89 ax.errorbar(dist, media, yerr=desvio, fmt="o", color="#1565C0",
90            ecolord="#1565C0", elinewidth=1.2, capsize=4, markersize=8,
91            zorder=5, label="Dados_reais_(media_+/-_desvio-padrao) ")
92
93 ax.set_xlabel("Distancia_do_AP_(m) ", fontsize=11)
94 ax.set_ylabel("RSSI_(dBm) ", fontsize=11)
95 ax.set_title("Comparacao:_dados_reais_x_modelo_teorico_log-distance\n"
96             "RSSI_em_funcao_da_distancia_(AP_2,4_GHz,_ESP32) ",
97             fontsize=11, fontweight="bold")
98 ax.grid(alpha=0.35)
99 ax.legend(fontsize=8.5, loc="upper_right")
100
101 # caixa de metricas
102 txt = (f"n_teorico=_({n_teorico:.1f})\n"
103       f"n_efetivo=_({n_efetivo:.2f})\n"
104       f"RMSE=_({rmse:.2f}_dB\n"
105       f"MAE=_({mae:.2f}_dB)")
106 ax.text(0.02, 0.04, txt, transform=ax.transAxes, fontsize=8.5,
107        va="bottom", ha="left",
108        bbox=dict(boxstyle="round", facecolor="#FFF8E1",
109                edgecolor="#E65100", alpha=0.9))
110
111 fig.tight_layout()
112 fig.savefig("images/comparativo.pdf")
113 fig.savefig("images/comparativo.png", dpi=150)
114 print("images/comparativo.pdf|png_gravado.")
115
116
117 if __name__ == "__main__":
118     main()

```

6 RESULTADOS

6.1 PARTE 1 — DADOS COLETADOS

Foram coletadas 270 amostras (9 distâncias $\times$ 30 amostras). A Tabela 2 resume a média, o desvio-padrão e os extremos do RSSI por distância; um excerto das amostras brutas é mostrado na Tabela 3. O conjunto completo encontra-se em `medicoes_rssi.csv`.

Tabela 2 – RSSI por distância — média, desvio-padrão e extremos (30 amostras).

Dist. (m)	Média (dBm)	Desvio (dB)	Mín (dBm)	Máx (dBm)	<i>n</i>
1,0	−40,03	1,33	−42	−37	30
1,5	−45,00	1,66	−49	−42	30
2,0	−49,30	2,09	−54	−46	30
2,5	−50,23	2,01	−54	−47	30
3,0	−55,37	2,09	−59	−52	30
3,5	−57,23	1,91	−62	−53	30
4,0	−60,80	3,11	−68	−56	30
4,5	−59,13	2,70	−65	−53	30
5,0	−62,50	3,36	−70	−58	30

Fonte: Elaborado pelos autores (2026).

Tabela 3 – Excerto do arquivo `medicoes_rssi.csv` (amostras a 1.0 m).

<code>distancia_m</code>	<code>amostra</code>	<code>rssi_dbm</code>
1.0	1	−39
1.0	2	−42
1.0	3	−41
1.0	4	−39
1.0	5	−40
...		

Fonte: Elaborado pelos autores (2026).

6.2 PARTE 2 — GRÁFICO DOS DADOS REAIS

A Figura 1 apresenta o RSSI médio em função da distância, com barras de desvio-padrão. Observa-se o decaimento esperado, com pequenas inflexões em 2.5 m e 4.5 m.

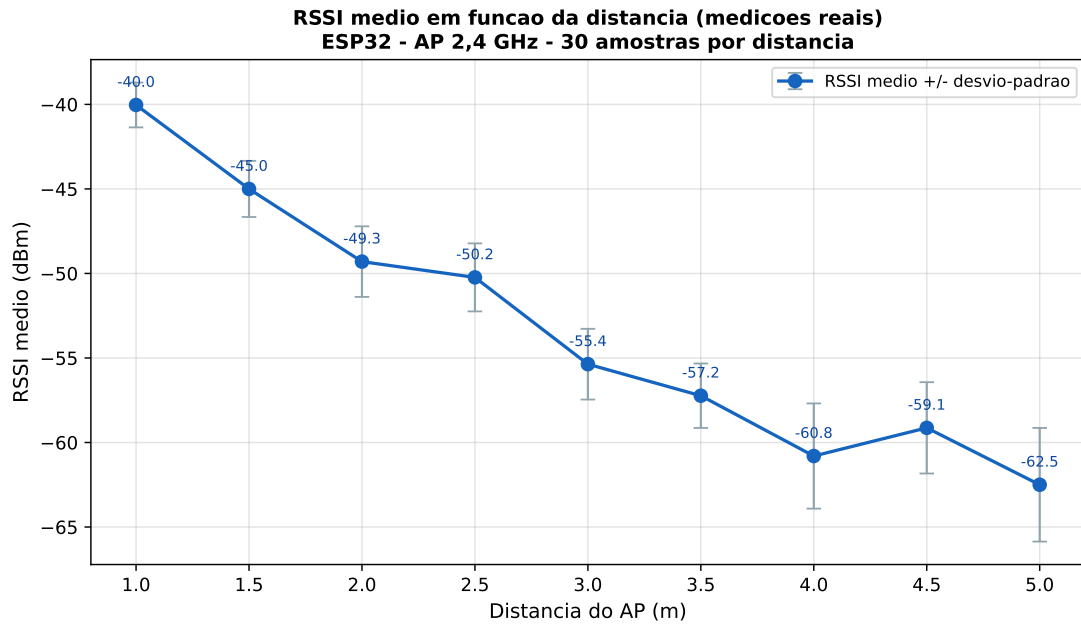


Figura 1 – RSSI médio medido em função da distância ao AP (30 amostras por distância).

Fonte: Elaborado pelos autores (2026).

6.3 PARTE 3 — SIMULAÇÃO TEÓRICA

A Figura 2 mostra as 30 amostras simuladas por distância, a média simulada e a curva teórica sem ruído, obtidas com $P_t = 20$ dBm, $n = 3,0$ e $\sigma = 4$ dB.

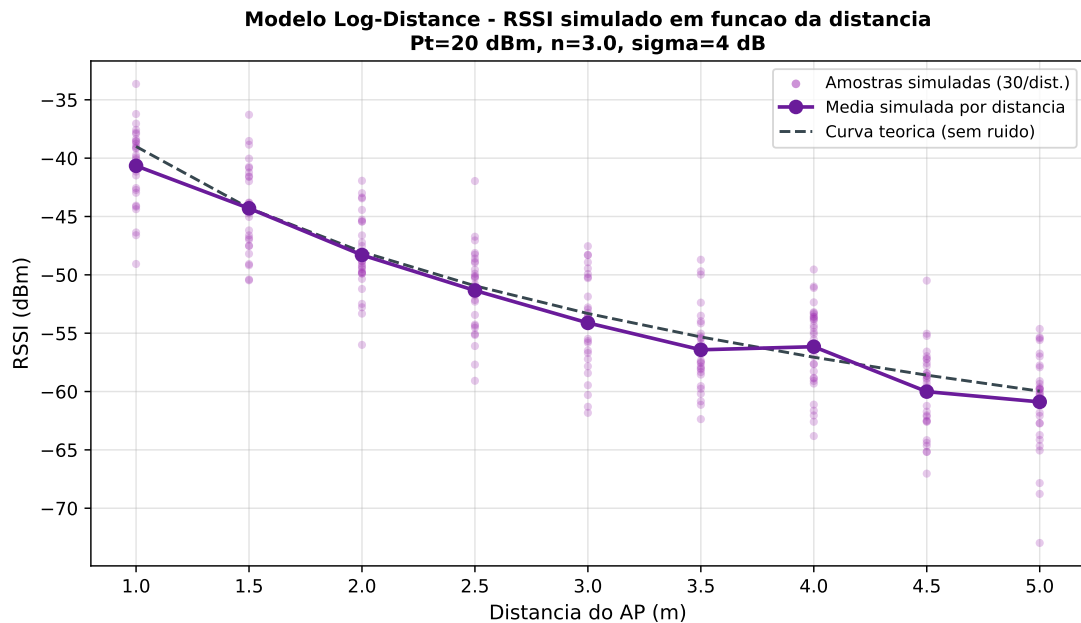


Figura 2 – RSSI simulado pelo modelo log-distance: amostras, média e curva teórica.

Fonte: Elaborado pelos autores (2026).

6.4 PARTE 4 — GRÁFICO COMPARATIVO

A Figura 3 sobrepõe os dados reais à curva teórica e à faixa de incerteza $\pm\sigma$. Todos os pontos reais permanecem dentro da faixa teórica. O ajuste por regressão sobre os dados reais forneceu expoente efetivo $n_{ef} = 3,23$ (vs. $n = 3,0$ teórico), com RMSE de 1.89 dB e MAE de 1.61 dB entre as médias.

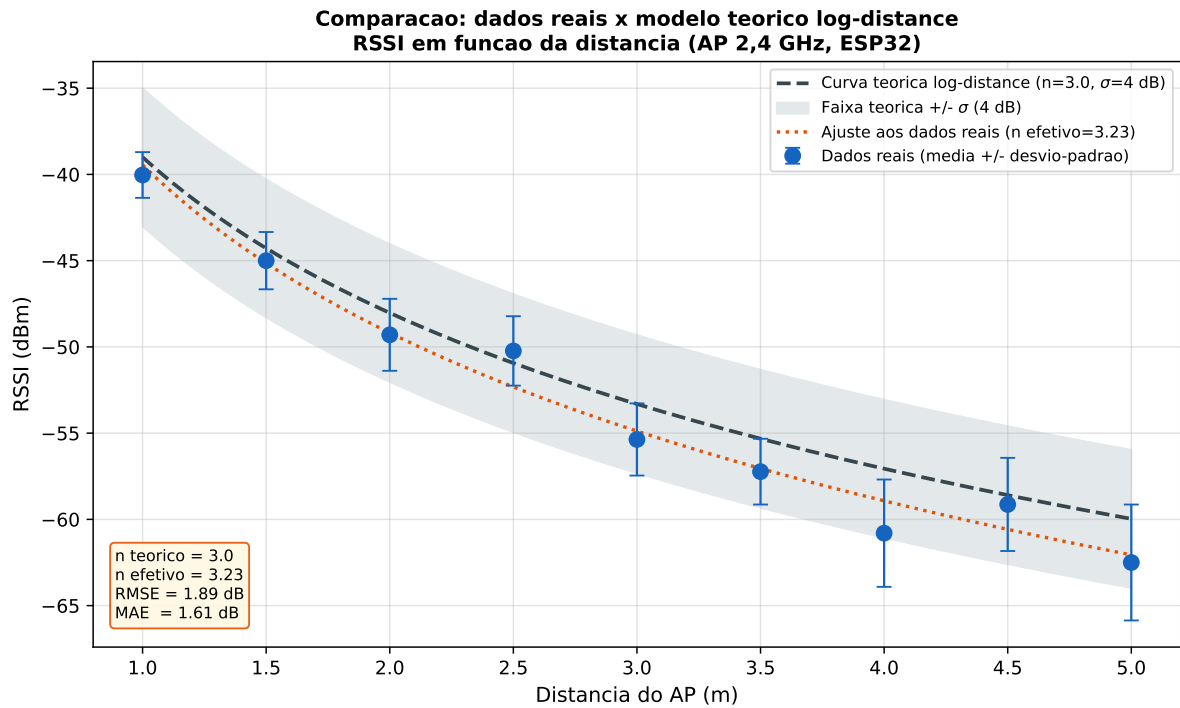


Figura 3 – Comparação entre os dados reais (pontos) e o modelo teórico log-distance (linha tracejada) com faixa $\pm\sigma$.

Fonte: Elaborado pelos autores (2026).

7 ANÁLISE E DISCUSSÃO

7.1 DIFERENÇAS ENTRE TEORIA E PRÁTICA

A aderência global entre as medidas e o modelo foi boa: o RMSE de 1.89 dB é da ordem de uma fração do desvio-padrão de *shadowing* adotado ($\sigma = 4$ dB), e todos os pontos reais caíram dentro da faixa $\pm\sigma$ (Figura 3). Ainda assim, o expoente de perda efetivo dos dados reais ($n_{ef} = 3,23$) foi superior ao valor teórico assumido ($n = 3,0$), indicando atenuação um pouco mais acentuada do que a prevista. As medidas mostraram-se sistematicamente ligeiramente abaixo da curva teórica a partir de 3.0 m, e não-monotônicas em 2.5 m e 4.5 m, onde o RSSI medido superou o do ponto anterior — comportamento incompatível com o decaimento puramente determinístico do modelo.

7.2 FATORES QUE EXPLICAM AS DISCREPÂNCIAS

- **Obstáculos e obstruções:** mobiliário e trechos de parede no percurso adicionam atenuação localizada (por exemplo, a queda acentuada em 4.0 m), elevando o expoente efetivo;
- **Reflexões e multipercurso:** a soma construtiva de componentes refletidas explica os “picos” de RSSI em 2.5 m e 4.5 m, em que o sinal medido foi melhor que o do ponto anterior;
- **Ruído e interferência cocanal:** a faixa de 2.4 GHz é densamente ocupada por redes vizinhas no ambiente residencial urbano, aumentando a variância das leituras (note-se o crescimento do desvio-padrão com a distância, de 1.33 dB a 3.36 dB);
- **Fatores de hardware:** a eficiência e o diagrama de irradiação das antenas do ESP32 e do AP, além da quantização inteira do RSSI, introduzem desvios adicionais;
- **Dinâmica do ambiente:** a movimentação de pessoas durante a coleta provoca *shadowing* variável no tempo.

7.3 IMPORTÂNCIA DA MODELAGEM ESTATÍSTICA

Os resultados evidenciam que um modelo puramente determinístico (termo $10 n \log_{10}(d/d_0)$) é insuficiente para representar o comportamento real do canal: sem o termo estocástico X_σ , não seria possível explicar nem a dispersão das 30 amostras em cada distância nem as inflexões não-monotônicas observadas. A modelagem estatística do *shadowing* via variável gaussiana de desvio σ é o que permite delimitar a faixa de incerteza ($\pm\sigma$) na qual as medidas efetivamente

ocorrem. Esse tratamento probabilístico é essencial para o dimensionamento de margens de enlace (*link budget*), para o planejamento de cobertura e, sobretudo, para sistemas de localização *indoor* baseados em RSSI, nos quais a incerteza do sinal precisa ser modelada explicitamente (1, 3).

8 CONCLUSÃO

A atividade cumpriu todos os objetivos propostos. Foram coletadas 270 amostras reais de RSSI com o ESP32 em nove distâncias (de 1.0 a 5.0 m), organizadas em CSV e reduzidas às médias por distância. O modelo log-distance foi implementado com parâmetros realistas ($P_t = 20$ dBm, $n = 3,0$, $\sigma = 4$ dB) e comparado aos dados reais em um único gráfico.

A comparação revelou boa concordância (RMSE de 1.89 dB; $n_{ef} = 3,23$), com discrepâncias atribuíveis a multipercurso, obstáculos, ruído e características de *hardware*. Conclui-se que o modelo log-distance reproduz adequadamente a tendência média de decaimento, desde que acompanhado do termo estocástico de *shadowing*, sem o qual não se representam as flutuações inerentes ao canal real. Como trabalho futuro, sugere-se estender a campanha a maiores distâncias e a múltiplos ambientes, bem como aplicar filtragem por média móvel para mitigar o ruído de pequena escala.

REFERÊNCIAS

- 1 BAHL, P.; PADMANABHAN, V. N. RADAR: An In-Building RF-based User Location and Tracking System. *Proceedings IEEE INFOCOM 2000*, v. 2, p. 775–784, 2000.
- 2 RAPPAPORT, T. S. *Wireless Communications: Principles and Practice*. 2. ed. Upper Saddle River: Prentice Hall, 2002.
- 3 GOLDSMITH, A. *Wireless Communications*. Cambridge: Cambridge University Press, 2005.
- 4 MOLISCH, A. F. *Wireless Communications*. 2. ed. Chichester: Wiley-IEEE Press, 2011.
- 5 Espressif Systems. *ESP32 Series Datasheet*. [S.l.], 2023. Disponível em: <https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf>.
- 6 Espressif Systems. *ESP32 WiFi API Reference*. [S.l.], 2023. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html>.
- 7 ANATEL. *Ato n. 14448/2017: Requisitos Técnicos para Equipamentos de Radiocomunicação de Radiação Restrita*. Brasília, 2017.